INFORME FINAL

Taller de Refactorización — Código Limpio

Sistema	Restaurante El Buen Sabor
Módulo	Sistema de Facturación
Referencia	Clean Code — Robert C. Martin

Programación de Computadores III · SS462 · Cuarto Semestre
Estudiante: Jose David De Jesús Reales Montero
Grupo: 02

1. Introducción

El presente informe documenta el proceso de refactorización del sistema Restaurante El Buen Sabor, un programa funcional pero desarrollado intencionalmente con múltiples malas prácticas de programación.

El objetivo del taller fue transformar este código en código limpio, manteniendo exactamente el mismo comportamiento, pero mejorando su legibilidad, mantenibilidad y diseño, siguiendo los principios descritos por Robert C. Martin en Clean Code.

2. Descripción del Problema Inicial

El sistema original presentaba un conjunto de problemas estructurales significativos que, aunque no impedían su funcionamiento, comprometían gravemente su sostenibilidad a largo plazo:

- Alto nivel de acoplamiento entre componentes
- Uso extensivo de variables globales (God Object)
- Métodos extensos con múltiples responsabilidades
- Duplicación de lógica en distintas partes del código
- Nombres poco descriptivos y variables de una sola letra
- Uso incorrecto de comentarios

> **Impacto en el Desarrollo Real**
> *Aunque el sistema funcionaba correctamente, su estructura dificultaba la comprensión y el mantenimiento, lo cual representa un riesgo significativo en escenarios reales de desarrollo de software.*

3. Proceso de Refactorización

La refactorización se realizó siguiendo cinco niveles definidos en la guía del taller, aplicados de forma progresiva para minimizar riesgos. Cada nivel construyó sobre el anterior, garantizando que el comportamiento del sistema se mantuviera intacto en todo momento.

#	Nivel	Enfoque Principal
Nivel 1	**Formato**	Indentación, llaves, espacios entre operadores

#	Nivel	Enfoque Principal
Nivel 2	**Nombres**	Variables, métodos y clases con nombres descriptivos
Nivel 3	**Estructura**	Eliminación de duplicados, constantes, responsabilidad única
Nivel 4	**Diseño**	Clases de dominio, POO real, eliminación de estado global
Nivel 5	**Comentarios**	Solo comentarios significativos, código autoexplicativo

3.1 Nivel 1 — Formato

En este nivel se mejoró la legibilidad visual del código sin alterar la lógica funcional:

- Indentación consistente de 4 espacios en todo el proyecto
- Estandarización del estilo de llaves (K&R style)
- Separación de instrucciones agrupadas en una sola línea
- Agregado de espacios entre operadores aritméticos y lógicos
- Corrección de todos los bloques if para usar llaves obligatorias

Este paso permitió una lectura clara y uniforme del código antes de modificar su estructura interna.

3.2 Nivel 2 — Nombres

Se renombraron variables, métodos y clases para que expresaran claramente su propósito dentro del dominio del negocio:

- Eliminación de nombres crípticos (x, tmp, d1, calc)
- Los métodos pasaron a describir exactamente lo que hacen (calcularTotalFactura, aplicarDescuento)
- Las clases comenzaron a representar conceptos del dominio real (Pedido, Factura, Producto)

Este nivel permitió que el código pudiera entenderse sin necesidad de comentarios explicativos adicionales.

3.3 Nivel 3 — Estructura

En este nivel se eliminaron duplicaciones y se mejoró la organización interna del código:

- Centralización del cálculo de la factura en una sola clase responsable
- Creación de constantes para los valores del negocio: IVA, descuento, propina
- División de métodos largos en métodos pequeños con responsabilidad única
- Separación clara entre la lógica de cálculo y la lógica de presentación/impresión

3.4 Nivel 4 — Diseño

Este fue el cambio más significativo y profundo del proyecto. El sistema migró de un diseño procedural a uno orientado a objetos real:

- Eliminación de arrays paralelos y del objeto global de datos
- Creación de clases de dominio: Producto, Pedido, Factura

- Encapsulamiento del estado dentro de objetos coherentes
- Aplicación de los principios de POO y del Principio de Responsabilidad Única (SRP)

> **Transformación Arquitectónica**
> *El sistema pasó de un diseño procedural con estado compartido a uno orientado a objetos real, donde cada clase tiene una identidad clara y sus responsabilidades están bien delimitadas.*

3.5 Nivel 5 — Comentarios

Finalmente, se revisaron y depuraron todos los comentarios del código:

- Eliminación de comentarios redundantes que repetían lo que el código ya expresaba
- Eliminación de comentarios de versiones antiguas y código comentado
- Conservación únicamente de comentarios que explican decisiones de diseño no evidentes

El resultado es un código preparado para explicarse por sí mismo, sin depender de anotaciones auxiliares.

4. Verificación del Criterio de Éxito

El sistema refactorizado fue evaluado contra los criterios de aceptación definidos al inicio del taller. Los resultados son los siguientes:

Criterio de Éxito	Estado
Resultados idénticos al sistema original	✔
Lectura clara y lineal	✔
Cada clase con responsabilidad única (SRP)	✔
Nombres descriptivos en todo el código	✔
Sin duplicación de lógica	✔
Comentarios mínimos y significativos	✔

Todos los criterios fueron satisfechos completamente. La verificación se realizó mediante comparación directa de los resultados numéricos del sistema original y del refactorizado, utilizando los mismos conjuntos de datos de entrada.

5. Conclusión

La refactorización permitió transformar un sistema difícil de mantener en un proyecto con diseño limpio, extensible y profesional. Cada nivel aplicado contribuyó de forma incremental a mejorar la calidad del código sin interrumpir su funcionamiento.

Este ejercicio demuestra cómo un código funcional puede convertirse en un problema serio si no se aplican buenas prácticas desde el inicio, y cómo la refactorización progresiva es la estrategia más segura y efectiva para la sostenibilidad del software a largo plazo.

> **Lección Clave**
> *El código limpio no es un lujo — es una inversión en la sostenibilidad del software. Un sistema funcional pero desordenado acumula deuda técnica que, eventualmente, se convierte en riesgo operacional real.*

Preguntas de Reflexión

Las siguientes respuestas corresponden al análisis crítico del proceso de refactorización, conectando la experiencia práctica del taller con implicaciones reales en el desarrollo de software profesional.

01

¿Cuánto tiempo tardaste en entender el código original? ¿Qué parte fue más difícil de descifrar?

Inicialmente tomó un tiempo considerable comprender el flujo del código, ya que la lógica estaba distribuida en múltiples métodos con nombres poco descriptivos y dependía fuertemente de variables globales. La parte más difícil de descifrar fue el cálculo de la factura, ya que la misma lógica estaba duplicada en varias clases y métodos, generando confusión sobre cuál era el cálculo "real".

02

¿Cuál de las 34 malas prácticas te pareció la más peligrosa en un sistema real? ¿Por qué?

La mala práctica más peligrosa es el uso de variables globales públicas (public static) concentradas en un objeto tipo God Object. En un sistema real, esto puede provocar errores difíciles de rastrear, ya que cualquier parte del programa puede modificar el estado global sin control, generando efectos colaterales impredecibles.

03

¿Qué pasaría si este sistema tuviera 50.000 líneas? ¿Cuánto costaría agregar un nuevo impuesto del 2%?

En el sistema original, agregar un nuevo impuesto implicaría modificar múltiples métodos y clases duplicadas, con alto riesgo de introducir errores. En el diseño refactorizado, bastaría con modificar una sola clase (Factura) y agregar una nueva constante, reduciendo drásticamente el costo y el riesgo del cambio.

04

Identifica un punto donde podrías haber introducido un bug al refactorizar. ¿Cómo lo detectarías?

Un punto crítico es la migración del cálculo desde una lógica duplicada hacia una lógica centralizada. Para detectar errores, se compararon los resultados numéricos del sistema original y del refactorizado con los mismos datos de entrada, verificando que las facturas produjeran exactamente los mismos valores.

05

¿Por qué los efectos secundarios ocultos son especialmente peligrosos en producción?

Los efectos secundarios ocultos dificultan la comprensión del comportamiento real del sistema, ya que un método puede modificar el estado global sin que su nombre lo indique. En producción, estos errores son difíciles de reproducir y diagnosticar, ya que dependen del orden de ejecución y del estado previo del sistema.

06

¿Cómo conectas este ejercicio con casos reales como Knight Capital, Southwest o Equifax?

Estos casos reales demuestran que el software falla no solo por errores lógicos, sino por diseños deficientes y código mal mantenido. Este ejercicio evidencia cómo un código aparentemente funcional puede convertirse en un riesgo crítico si no se refactoriza adecuadamente a tiempo.